

Plan de Branching

Estrategia Git Flow Adaptada — Donaton Parcial 2

DSY1106 Desarrollo Fullstack III | DuocUC

Integrantes	Oscar Flores · Leroy Rodriguez
Profesor	Juan Valentin Mora Ruiz
Estrategia	Git Flow Adaptado (2 integrantes, sprint 8 horas)

1. Estrategia Seleccionada: Git Flow Adaptado

Se implementa una versión simplificada de **Git Flow**, adaptada para un equipo de 2 personas y un sprint de 8 horas. La estrategia mantiene las ramas principales **main** y **develop**, con ramas **feature** por componente y subcomponente.

2. Estructura de Ramas

```
main
├── develop
│   ├── feature/ms-donaciones
│   │   ├── feature/ms-donaciones-repository
│   │   ├── feature/ms-donaciones-factory
│   │   └── feature/ms-donaciones-tests
│   ├── feature/ms-logistica
│   │   ├── feature/ms-logistica-repository
│   │   ├── feature/ms-logistica-observer
│   │   └── feature/ms-logistica-tests
│   ├── feature/bff-donaton
│   │   ├── feature/bff-service
│   │   └── feature/bff-tests
│   ├── feature/frontend-donaton
│   └── feature/arquetipos-maven
```

3. Descripción de Ramas

Rama	Propósito	Responsable
main	Código estable para evaluación. Solo recibe merges desde develop.	Ambos
develop	Integración continua. Base para todas las features.	Ambos
feature/ms-donaciones	MS-Donaciones completo.	Oscar Flores
feature/ms-donaciones-repository	Repository Pattern.	Oscar Flores
feature/ms-donaciones-factory	Factory Method para tipos de donación.	Oscar Flores
feature/ms-donaciones-tests	Pruebas unitarias con Mockito.	Oscar Flores

feature/ms-logistica	MS-Logística completo.	Leroy Rodriguez
feature/ms-logistica-repository	Repository Pattern (Centros + Envíos).	Leroy Rodriguez
feature/ms-logistica-observer	Observer Pattern.	Leroy Rodriguez
feature/ms-logistica-tests	Pruebas unitarias LogisticaServiceTest.	Leroy Rodriguez
feature/bff-donaton	BFF que agrega ambos microservicios.	Leroy Rodriguez
feature/frontend-donaton	Frontend React (Dashboard + DonacionForm).	Oscar Flores
feature/arquetipos-maven	Arquetipo Maven para nuevos microservicios.	Ambos

4. Flujo de Trabajo

4.1 Crear y Trabajar en una Rama Feature

```
# 1. Partir siempre desde develop actualizado
git checkout develop
git pull origin develop
git checkout -b feature/ms-donaciones

# 2. Commits atómicos con convención semántica
git add src/main/java/donaton/msdonaciones/repository/
git commit -m "feat(ms-donaciones): implementar DonacionRepository con Repository Pattern"

git add src/main/java/donaton/msdonaciones/factory/
git commit -m "feat(ms-donaciones): agregar Factory Method para tipos de donación"

git add src/test/
git commit -m "test(ms-donaciones): pruebas unitarias DonacionServiceTest con mocks Mockito"
```

4.2 Integrar Feature en develop

```
# Merge con --no-ff para preservar historial de feature
git checkout develop
git merge --no-ff feature/ms-donaciones \
  -m "merge: integrar ms-donaciones con Repository Pattern y Factory Method"
git push origin develop
```

4.3 Resolución de Conflictos — Ejemplo Real

Durante la integración de **feature/bff-donaton** con **develop**, se produjo un conflicto en **application.properties** porque ambas ramas modificaron el puerto del servidor:

```
<<<<<< HEAD (develop - ms-logistica)
server.port=8082
=====
server.port=8080
>>>>>> feature/bff-donaton
```

Resolución: Se acordó que el BFF usa el puerto 8080 y MS-Logística el 8082. Se editó manualmente el archivo conflictivo y se realizó el commit de resolución:

```
git add src/main/resources/application.properties
git commit -m "fix(merge): resolver conflicto de puertos - BFF:8080, MS-Logistica:8082"
```

4.4 Release hacia main

```
git checkout main
git merge --no-ff develop \
  -m "release: v1.0.0 Parcial 2 - BFF + 2 microservicios + 3 patrones de diseño"
git tag -a v1.0.0 -m "Parcial 2 entregado - Factory Method, Repository Pattern, Observer"
git push origin main --tags
```

5. Convención de Commits

Prefijo	Uso	Ejemplo
feat	Nueva funcionalidad	feat(ms-donaciones): agregar Factory Method
test	Pruebas unitarias	test(ms-logistica): pruebas Observer con mocks
fix	Corrección de errores	fix(merge): resolver conflicto de puertos
docs	Documentación	docs: agregar README ms-donaciones
refactor	Refactorización	refactor(bff): extraer lógica de conteo a método privado
merge	Commit de resolución	merge: integrar feature/ms-logistica en develop

6. Cómo la Estrategia Favoreció la Colaboración

- Trabajo paralelo sin conflictos: Oscar desarrolló MS-Donaciones y el frontend mientras Leroy desarrollaba MS-Logística y el BFF, cada uno en su propia rama feature/ sin interferir.
- Integración controlada: solo se fusiona a develop cuando la feature está completa y probada, evitando que código incompleto rompa el trabajo del otro integrante.
- Trazabilidad: cada commit tiene un prefijo semántico que identifica claramente qué cambió y en qué componente del sistema.
- Resolución documentada: los conflictos se resuelven con commits explicativos que dejan trazabilidad en el historial del repositorio.
- Historial limpio: el uso de --no-ff en los merges preserva las ramas feature en el historial, permitiendo ver exactamente qué commits corresponden a cada funcionalidad.